

Imbalanced Classification

Practical Methods

Sourav Karmakar

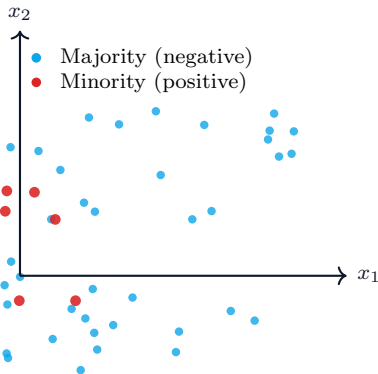
Outline

- 1 Introduction & Motivation
- 2 Random Oversampling
- 3 SMOTE
- 4 Random Undersampling
- 5 Class Weights & Loss Adjustment
- 6 Class Weight Parameters in Popular Models
- 7 Evaluation: ROC-AUC vs. PR-AUC
- 8 Quick Comparison & Tips

1. Introduction & Motivation

What Is Class Imbalance?

- The **positive (minority)** class is far rarer than the negative (majority) class.
- Examples: fraud detection, disease diagnosis, defect identification, churn prediction.
- A naïve classifier predicting “always negative” can achieve high accuracy yet miss every positive.



Why Accuracy Is Misleading

Motivating Example

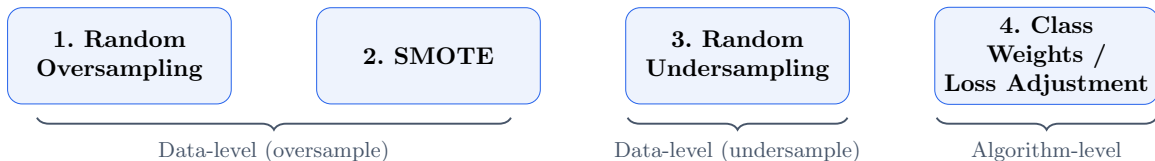
Dataset: 9,950 negatives vs. 50 positives ($N = 10,000$).

A model predicting *always negative* achieves **99.5% accuracy** but **0% recall**.

Better metrics for imbalanced settings:

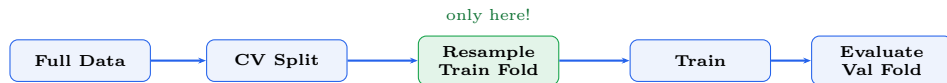
- **Recall (Sensitivity):** $\frac{TP}{TP + FN}$
- **Precision:** $\frac{TP}{TP + FP}$
- **F_β score:** $(1 + \beta^2) \frac{\text{Precision} \cdot \text{Recall}}{\beta^2 \text{Precision} + \text{Recall}}$
 - $\beta < 1$: emphasises Precision.
 - $\beta > 1$: emphasises Recall.
- **ROC-AUC** and **PR-AUC** (discussed later).

Four Core Techniques — Overview



Key Principle — Resample Inside CV Folds

If you resample *before* splitting, duplicated (or synthetic) minority samples can leak into the validation fold, giving you falsely optimistic scores. Always split first, then resample only the training fold.

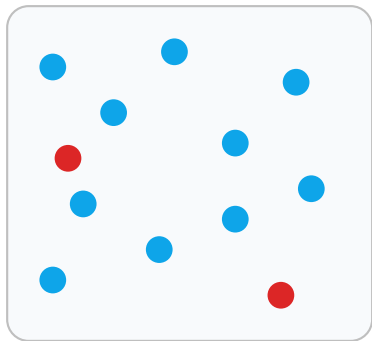


2. Random Oversampling

Random Oversampling — Idea

Duplicate minority samples (at random, with replacement) until the class counts are balanced.

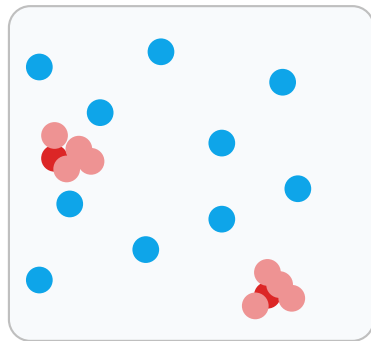
Before



Duplicate



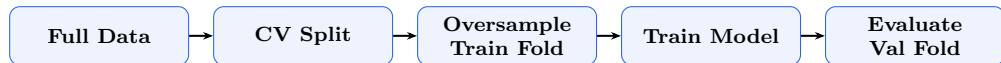
After oversampling



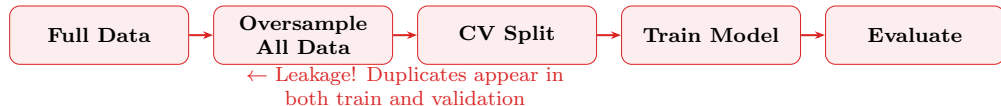
Practical Tips for Random Oversampling

- 1 Combine with **data augmentation** (add small noise / jitter) to reduce overfitting risk.
- 2 Always apply oversampling **inside** each CV fold—never on the full dataset.
- 3 Use as a **baseline** to compare against SMOTE and other variants.
- 4 Consider light oversampling (e.g., $2\times$ – $5\times$) rather than exact balance.

Correct:



Wrong:



3. SMOTE

Synthetic Minority Oversampling Technique

Generate **synthetic** minority samples by interpolating between existing minority points.

Steps:

- 1 Pick a minority point $\mathbf{x}_i$.
- 2 Find its k -nearest minority neighbours.
- 3 Randomly choose one neighbour $\mathbf{x}_z$.
- 4 Draw $\lambda \sim \mathcal{U}(0, 1)$ and create:

$$\mathbf{x}_{\text{new}} = \mathbf{x}_i + \lambda (\mathbf{x}_z - \mathbf{x}_i)$$

- 5 Repeat until the desired oversampling ratio is reached.

SMOTE — Mathematical Formulation

Given minority set $\mathcal{X}_{\min} = \{\mathbf{x}_1, \dots, \mathbf{x}_m\}$:

- 1 Choose $\mathbf{x}_i \in \mathcal{X}_{\min}$.
- 2 Select neighbour $\mathbf{x}_z \in \text{NN}_k(\mathbf{x}_i) \cap \mathcal{X}_{\min}$.
- 3 Sample:

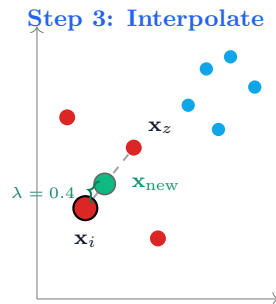
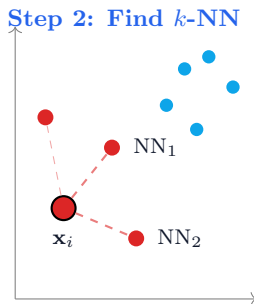
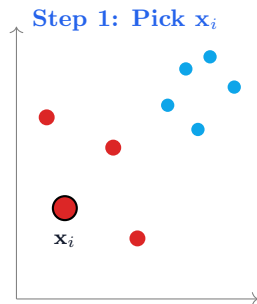
$$\mathbf{x}_{\text{new}} = \mathbf{x}_i + \lambda (\mathbf{x}_z - \mathbf{x}_i), \quad \lambda \sim \mathcal{U}(0, 1).$$

1-D Example:

If $x_i = 2.0$ and $x_z = 5.0$ with $\lambda = 0.3$:

$$x_{\text{new}} = 2.0 + 0.3 (5.0 - 2.0) = 2.0 + 0.9 = \mathbf{2.9}$$

SMOTE — Step-by-Step Visualisation



● Majority ● Minority ● Synthetic
Synthetic

The green point x_{new} lies on the segment between x_i and its chosen neighbour x_z , controlled by λ .

SMOTE — Pros, Cons & Variants

Pros

- Creates genuinely **new** feature combinations, so the model sees more diversity than exact copies.
- Smooths decision regions.
- Supported by all major ML frameworks (scikit-learn, etc.).

Cons

- Can create ambiguous samples near class boundaries.
- Sensitive to feature scaling.
- Not designed for categorical features.

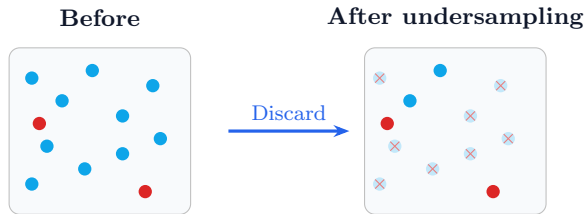
Popular Variants

- **Borderline-SMOTE:** Only oversample minority points near the decision boundary.
- **SMOTE-NC:** Handles mixed numerical + categorical features.
- **ADASYN:** Adaptively generates more synthetics for harder-to-learn minorities.
- **SMOTE + Tomek Links:** Combine oversampling with cleaning overlapping pairs.

4. Random Undersampling

Random Undersampling — Idea

Remove a subset of majority samples to reduce imbalance.



Pros

- Faster training (smaller dataset).
- Simplifies the decision boundary.
- Useful when majority data is very redundant.

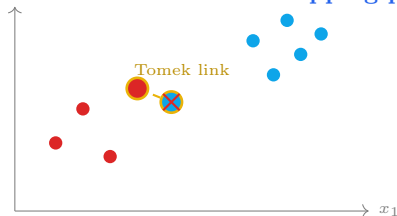
Cons

- Risk of discarding **informative** majority samples.
- Higher variance if too aggressive.
- Can lose subtle patterns in majority class.

Undersampling — Practical Notes

- 1 Prefer **informed** strategies: keep “hard negatives” near the boundary.
- 2 Combine with cost-sensitive loss rather than extreme undersampling.
- 3 **Tomek Links**: remove majority samples that form closest pairs with minority samples (cleans boundary).
- 4 Always apply inside CV folds to avoid leakage.

Tomek Links — Remove overlapping pairs



5. Class Weights & Loss Adjustment

Weighted Binary Cross-Entropy

Modify the loss so that minority-class errors cost **more**.

Let $y_i \in \{0, 1\}$, $p_i = \Pr(y_i = 1 \mid \mathbf{x}_i)$, weights w_1 (positives) and w_0 (negatives):

$$\mathcal{L}_{\text{wBCE}} = -\frac{1}{N} \sum_{i=1}^N \left[w_1 y_i \log p_i + w_0 (1 - y_i) \log(1 - p_i) \right]$$

Inverse-frequency weighting (common choice): for class c with count n_c , total n , and C classes:

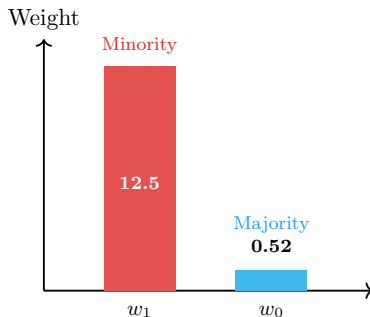
$$w_c = \frac{n}{C \cdot n_c}$$

Effect: Larger $w_1 \Rightarrow$ steeper gradient for false negatives $\Rightarrow$ improved recall (at the possible cost of precision). **Tune the decision threshold after training.**

Class Weights — Worked Example

Dataset: 200 positives, 4,800 negatives ($n = 5,000$, $C = 2$).

$$w_1 = \frac{5000}{2 \times 200} = 12.5 \qquad w_0 = \frac{5000}{2 \times 4800} \approx 0.52$$



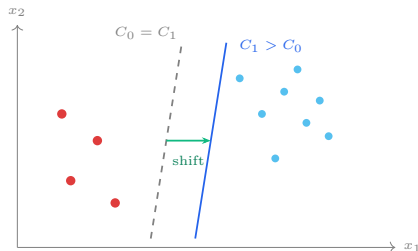
The optimizer pays $\sim 24\times$ more attention to minority errors.

Margin-Based Models (SVM)

For a linear SVM with hinge loss, class-specific penalty C_c scales the slack:

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + \sum_{i=1}^N C_{y_i} \xi_i \\ \text{s.t.} \quad & y_i (\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0. \end{aligned}$$

Setting $C_1 > C_0$ makes positive-class violations more costly, shifting the boundary toward the majority class.



6. Class Weight Parameters in Popular Models

Logistic Regression

```
from sklearn.linear_model import
    LogisticRegression

model = LogisticRegression(
    class_weight="balanced"    # or {0:1,
                               1:12.5}
)
model.fit(X_train, y_train)
```

Random Forest

```
from sklearn.ensemble import
    RandomForestClassifier

model = RandomForestClassifier(
    class_weight="balanced_subsample",
    n_estimators=200
)
model.fit(X_train, y_train)
```

Decision Tree

```
from sklearn.tree import
    DecisionTreeClassifier

model = DecisionTreeClassifier(
    class_weight="balanced"
)
model.fit(X_train, y_train)
```

SVM (SVC)

```
from sklearn.svm import SVC

model = SVC(
    class_weight="balanced",
    kernel="rbf"
)
model.fit(X_train, y_train)
```

Python Code Examples — Boosting Models

XGBoost

```
from xgboost import XGBClassifier
import numpy as np

neg, pos = np.bincount(y_train)

model = XGBClassifier(
    scale_pos_weight=neg / pos
)
model.fit(X_train, y_train)
```

LightGBM

```
from lightgbm import LGBMClassifier

# Option A: automatic balancing
model = LGBMClassifier(is_unbalance=True)

# Option B: manual weight
model = LGBMClassifier(
    scale_pos_weight=neg / pos
)
model.fit(X_train, y_train)
```

CatBoost

```
from catboost import CatBoostClassifier

# Option A: auto balanced
model = CatBoostClassifier(
    auto_class_weights="Balanced"
)

# Option B: manual weights
model = CatBoostClassifier(
    class_weights=[0.52, 12.5]
)
model.fit(X_train, y_train)
```


Class Weight Parameters — Reference Table

Model	Parameter	Options	Default
LogisticRegression	<code>class_weight</code>	None, "balanced", dict	None
DecisionTreeClassifier	<code>class_weight</code>	None, "balanced", dict	None
RandomForestClassifier	<code>class_weight</code>	None, "balanced", "balanced_subsample", dict	None
SVC / LinearSVC	<code>class_weight</code>	None, "balanced", dict	None
XGBClassifier	<code>scale_pos_weight</code>	Positive float ($n_{\text{neg}}/n_{\text{pos}}$)	1
LGBMClassifier	<code>is_unbalance</code>	True / False	False
	<code>scale_pos_weight</code>	Positive float	1.0
	<code>class_weight</code>	"balanced", dict (multi-class)	None
CatBoostClassifier	<code>class_weights</code>	List of floats	None
	<code>auto_class_weights</code>	"Balanced", "SqrtBalanced"	None

"balanced" vs. "balanced_subsample"

balanced: $w_c = \frac{n}{C \cdot n_c}$ computed on the full training set.

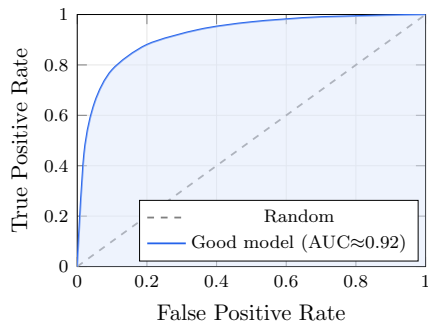
balanced_subsample: Same formula but recomputed *per bootstrap sample* (ensembles).

7. Evaluation: ROC-AUC vs. PR-AUC

ROC Curve

The **Receiver Operating Characteristic** curve plots True Positive Rate vs. False Positive Rate at various thresholds.

$$\text{TPR (Recall)} = \frac{TP}{TP + FN} \qquad \text{FPR} = \frac{FP}{FP + TN}$$

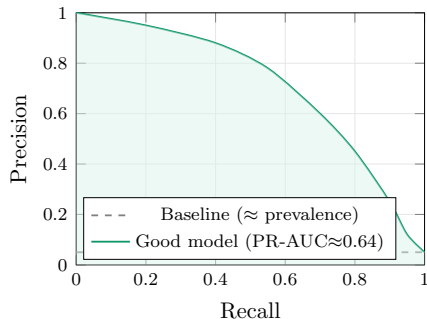


PR Curve — Better for Imbalanced Data

The **Precision–Recall** curve plots Precision vs. Recall at various thresholds.

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$



ROC-AUC vs. PR-AUC — When to Use Which?

ROC-AUC

- Considers both classes equally.
- Can look overly optimistic when negatives vastly outnumber positives.
- Good for balanced or mildly imbalanced settings.

**Severe
imbalance?**

→
Prefer PR-AUC

PR-AUC

- Focuses on the minority (positive) class.
- More informative under severe imbalance.
- Baseline = class prevalence (not 0.5).

Rule of Thumb

If your positive class is $< 5\text{--}10\%$ of the data, **PR-AUC** is generally more informative than ROC-AUC for model selection.

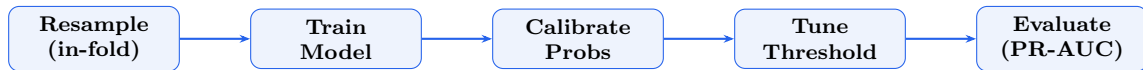
8. Quick Comparison & Tips

Method Comparison

	Oversampling	SMOTE	Undersampling	Class Weights
New information	No	Yes	No	N/A
Overfitting risk	High	Medium	Low	Low
Training speed	Slower	Slower	Faster	Same
Model-agnostic	Yes	Yes	Yes	Needs support
Handles cat. features	Yes	Special variant	Yes	Yes

Evaluation Checklist

- 1 Use **stratified** k -fold CV; maintain time order for temporal data.
- 2 Prefer **PR-AUC** and *recall @ target precision* for rare positives.
- 3 Tune the **decision threshold** on a held-out validation set aligned to business costs.
- 4 Guard against **leakage**: perform all resampling *inside* each CV fold.
- 5 Consider **probability calibration** (Platt scaling / isotonic regression) after training.
- 6 Combine methods: e.g., light SMOTE + class weights + threshold tuning.



Thank You!

Questions?